

DATA STRUCTURES

Complete Technical Course Handbook & Detailed Syllabus Notes

Teaching Scheme: 3-1-0 (L-T-P)

Coverage: Units I through V (End-to-End)

Focus: Algorithms, C Code, & Complexities

UNIT I: INTRODUCTION, ARRAYS & LINKED LISTS

1. Introduction & Basic Terminology

A **Data Structure** is a specialized format for organizing, processing, retrieving, and storing data. It provides a means to manage large amounts of data efficiently for uses such as large databases and internet indexing services. In the C programming language, basic built-in data types (like `int`, `float`, `char`) are used to construct more complex user-defined structures.

An **Abstract Data Type (ADT)** is a mathematical model for data types, where a data type is defined by its behavior (semantics) from the point of view of a user of the data, specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations. ADTs separate the logical properties from implementation details.

Algorithms and Complexity

- **Algorithm:** A finite set of instructions that, if followed, accomplishes a particular task. Characteristics include Input, Output, Definiteness, Finiteness, and Effectiveness.
- **Time Complexity:** The amount of computer time it needs to run to completion. Usually estimated by counting the number of elementary operations performed.
- **Space Complexity:** The amount of memory space required by the algorithm during its execution, including fixed part (code space, simple variables) and variable part (dynamic allocations, recursion stack).
- **Time-Space Trade-off:** The reality that an algorithm can often be made faster by using more memory, or consume less memory by running slower.

Asymptotic Notations

Used to mathematically describe the limiting behavior of a function when the argument tends towards a particular value or infinity. Let $f(n)$ and $g(n)$ be functions mapping non-negative integers to real numbers.

- **Big Oh (O):** Asymptotic upper bound. The worst-case scenario. $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- **Big Omega (Ω):** Asymptotic lower bound. The best-case scenario. $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.
- **Big Theta (Θ):** Asymptotic tight bound. Average-case scenario. $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

2. Arrays

An **Array** is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together, allowing calculation of the position of each element by simply adding an offset to a base value (the address of the first element).

Derivation of Index Formulae

Assuming $Base$ is the starting memory address, w is the word size in bytes, and array bounds range from L (lower bound) to U (upper bound).

• 1-D Array:

$$Loc(A[i]) = Base + w \cdot (i - L)$$

- **2-D Array (Row Major Order):** Elements are stored row by row. If the array is defined as $A[L_r \dots U_r][L_c \dots U_c]$, and N is the number of columns ($U_c - L_c + 1$):

$$Loc(A[i][j]) = Base + w \cdot [N \cdot (i - L_r) + (j - L_c)]$$

- **2-D Array (Column Major Order):** Elements are stored column by column. If M is the number of rows ($U_r - L_r + 1$):

$$Loc(A[i][j]) = Base + w \cdot [(i - L_r) + M \cdot (j - L_c)]$$

- **3-D Array (Row Major Order):** For array $A[L_1 \dots U_1][L_2 \dots U_2][L_3 \dots U_3]$, where $E_2 = (U_2 - L_2 + 1)$ and $E_3 = (U_3 - L_3 + 1)$:

$$Loc(A[i][j][k]) = Base + w \cdot [E_2 \cdot E_3 \cdot (i - L_1) + E_3 \cdot (j - L_2) + (k - L_3)]$$

Sparse Matrices

A matrix in which the majority of the elements are zero. It is highly inefficient to use a standard 2D array because it wastes memory and processing time on zeros.

Representations:

- *Array Representation (Triplets)*: A 2D array with 3 columns storing Row, Column, and Value. The 0th row usually stores Total Rows, Total Columns, and Total Non-Zero Values.
- *Linked List Representation*: Each node contains Row, Col, Value, and a pointer to the next node.

3. Linked Lists

A **Linked List** is a linear data structure in which elements are not stored at contiguous memory locations. Each element (node) contains a data field and a reference (pointer) to the next node. This allows for dynamic memory allocation and efficient insertions/deletions.

Types of Linked Lists

- **Singly Linked List**: Navigation is forward only.
- **Doubly Linked List**: Each node contains a pointer to the previous node and the next node. Allows bidirectional traversal but requires more memory per node.
- **Circular Linked List**: The `next` pointer of the last node points back to the first node. Can be singly or doubly circular.

```

// Implementation of Singly Linked List Insertion and Deletion in C
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Insert at the End
void insertEnd(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    struct Node* last = *head_ref;
    new_node->data = new_data;
    new_node->next = NULL;

    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    while (last->next != NULL)
        last = last->next;
    last->next = new_node;
}

// Delete by Key
void deleteNode(struct Node** head_ref, int key) {
    struct Node *temp = *head_ref, *prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head_ref = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}

```

Polynomial Representation and Operations

A polynomial like $P(x) = 4x^3 + 3x^2 + 2$ can be easily represented using a linked list where each node contains: **coefficient**, **exponent**, and **next pointer**.

Addition of Two Polynomials: Traverse both lists. If exponent of list 1 > list 2, append list 1's term to result and advance list 1. If equal, add coefficients. If list 1 < list 2, append list 2's term and advance list 2.

UNIT II: STACKS AND QUEUES

1. Stacks

A **Stack** is a linear ADT following the LIFO (Last In First Out) principle. The insertion and deletion happen at the exact same end, known as the **Top** of the stack.

Array and Linked Implementation

Array Implementation: Uses a static array of size **MAX**. The **top** index is initialized to -1. Push increments **top** and stores the value. Pop returns the value and decrements **top**. Risk of Stack Overflow.

Linked Implementation: Uses a singly linked list. **top** is a pointer to the head node. Push inserts at the beginning ($O(1)$). Pop deletes from the beginning ($O(1)$). No overflow unless memory is exhausted.

Applications of Stack: Polish Notations

Stacks are heavily used by compilers to evaluate expressions and manage syntax parsing.

- **Infix:** Operator is between operands ($A + B$).
- **Postfix (Reverse Polish Notation):** Operator follows operands ($A B +$).
- **Prefix (Polish Notation):** Operator precedes operands ($+ A B$).

Evaluation of Postfix Expression:

1. Scan the expression from left to right.
2. If the token is an operand, push it onto the stack.
3. If the token is an operator, pop two operands (Top is A , Next to Top is B). Evaluate $B \text{ ext{ operator } } A$. Push the result back to the stack.
4. When the string ends, the stack contains the final result.

Iteration and Recursion

- **Principles of Recursion:** A function calling itself. Requires a base condition to terminate. It uses the system call stack, which means it has $O(n)$ space complexity for n recursive calls.
- **Tail Recursion:** A function is tail-recursive if the recursive call is the very last operation in the function. Compilers can optimize this to prevent stack frames from accumulating, making it run in $O(1)$ auxiliary space.
- **Tower of Hanoi:** Move n disks from source peg to destination peg using an auxiliary peg. Rule: Only one disk moved at a time, no larger disk on top of a smaller disk. Time complexity: $O(2^n)$. Recurrence relation: $T(n) = 2T(n-1) + 1$.
- **Iteration vs Recursion Trade-offs:** Iteration is faster and more memory-efficient. Recursion provides cleaner, mathematically elegant code for complex problems like tree traversals and divide-and-conquer algorithms.

2. Queues

A **Queue** is a linear ADT following the FIFO (First In First Out) principle. Elements are inserted at the **Rear** and removed from the **Front**.

Types of Queues

- **Linear Queue:** Insertion at rear, deletion at front. *Problem:* False Overflow. Once the rear reaches the end of the array, no more elements can be inserted even if the front has been dequeued and there is empty space.
- **Circular Queue:** Overcomes false overflow by connecting the last index back to the first.

$$Rear = (Rear + 1) \% MAX$$

$$Front = (Front + 1) \% MAX$$

Condition for Full: $(Rear + 1) \% MAX == Front$

- **Deque (Double Ended Queue):** Elements can be inserted and removed from both ends. Can be Input-Restricted (insertion at one end, deletion at both) or Output-Restricted (insertion at both, deletion at one end).
- **Priority Queue:** Each element has a priority. Elements are dequeued based on priority rather than arrival time. Typically implemented using Binary Heaps for optimal $O(\log n)$ insertion and deletion.

UNIT III: SEARCHING, HASHING & SORTING

1. Searching

- **Sequential (Linear) Search:** Traverses array sequentially. Average and Worst case: $O(n)$. Suitable for small or unsorted datasets.
- **Binary Search:** Requires a sorted array. Divides the search space in half repeatedly. Average and Worst case: $O(\log n)$.
- **Index Sequential Search:** Data is sorted. An index file is created holding keys and pointers to data blocks. The search looks up the index file first (using linear or binary search) to locate the block, then searches the block linearly.

2. Hashing and Collision Resolution

Hashing transforms a key into an array index using a hash function, ideally offering $O(1)$ lookup time.

Collision Resolution Techniques

A collision happens when $h(k_1) = h(k_2)$ for $k_1 \neq k_2$.

- **Chaining:** Each array slot is a pointer to a linked list. Colliding elements are appended to the list. Load factor $\alpha = n/m$. Expected search time is $O(1 + \alpha)$.
- **Open Addressing:** All elements stay in the array. When collision occurs, probe for the next empty slot.
 - **Linear Probing:** $h(k, i) = (h'(k) + i) \% m$. Suffers from primary clustering.
 - **Quadratic Probing:** $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \% m$. Avoids primary clustering but suffers from secondary clustering.
 - **Double Hashing:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \% m$. Best open addressing method.

3. Sorting Algorithms

Arranging data in a specific (e.g., ascending) order.

Algorithm	Best Case Time	Worst Case Time	Space / Stability / Paradigm
Bubble Sort	$O(n)$	$O(n^2)$	$O(1)$ / Stable / In-place swap
Insertion Sort	$O(n)$	$O(n^2)$	$O(1)$ / Stable / Good for small N
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$ / Unstable / Min-swaps
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$ / Stable / Divide & Conquer
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$ / Unstable / Divide & Conquer
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$ / Unstable / Selection via Heap
Radix Sort	$O(nk)$	$O(nk)$	$O(n+k)$ / Stable / Non-comparative

Quick Sort Algorithm Details: Select a pivot element. Partition the array such that elements smaller than pivot go left, and greater go right. Recursively apply to left and right sub-arrays. Worst case $O(n^2)$ occurs when the array is already sorted and the pivot is the smallest/largest element.

UNIT IV: TREES

1. Basic Terminology & Tree Types

A **Tree** is a non-linear data structure representing hierarchical data.

Root: Top node. *Leaf:* Node with no children. *Depth:* Edges from root to node. *Height:* Longest path from node to leaf.

- **Binary Tree:** A tree where each node has at most 2 children.
 - *Strictly (Full) Binary Tree:* Every node has 0 or 2 children.
 - *Complete Binary Tree:* All levels are completely filled except possibly the last, which is filled from left to right.
 - *Extended Binary Tree (2-Tree):* All null pointers are replaced with special external nodes.
- **Tree Traversal Algorithms:**
 - *Inorder:* Traverse Left Subtree $\rightarrow$ Visit Node $\rightarrow$ Traverse Right Subtree.
 - *Preorder:* Visit Node $\rightarrow$ Traverse Left Subtree $\rightarrow$ Traverse Right Subtree.
 - *Postorder:* Traverse Left Subtree $\rightarrow$ Traverse Right Subtree $\rightarrow$ Visit Node.

2. Binary Search Tree (BST)

A binary tree where the left child's value is less than the parent's, and the right child's value is greater. Inorder traversal yields sorted data.

```
// BST Insertion in C
struct Node* insert(struct Node* node, int key) {
    if (node == NULL) return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    return node;
}
```

Search, Insert, and Delete operate in $O(h)$ time, where h is the height of the tree. In the worst case (skewed tree), $h = n$, making operations $O(n)$.

3. Advanced Trees

- **Threaded Binary Trees:** Space optimization where null left/right pointers in leaf nodes are replaced with "threads" pointing to the node's Inorder Predecessor / Successor. This enables inorder traversal without using recursion or a stack.
- **Huffman Coding:** A greedy algorithm used for lossless data compression. It builds a variable-length prefix-free binary tree where the most frequent characters have the shortest paths from the root.
- **AVL Tree:** A self-balancing BST. It maintains a **Balance Factor** $BF = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$. BF must be -1 , 0 , or 1 . If violated after insertion/deletion, it uses LL, RR, LR, or RL rotations to rebalance, guaranteeing $O(\log n)$ search time.
- **B-Tree:** A generalized m-way search tree where nodes can have more than one key and more than two children. Extensively used in databases and file systems due to shallow height, minimizing disk I/O.
- **Binary Heap:** A Complete Binary Tree typically represented as an array (where left child is $2i+1$ and right is $2i+2$). Can be Max-Heap or Min-Heap. Fundamental for Heap Sort and Priority Queues.

UNIT V: GRAPHS

1. Graph Terminology and Representation

A **Graph** is a pair $G = (V, E)$, where V is a set of vertices (nodes) and E is a set of edges connecting the vertices. Graphs can be Directed/Undirected, Weighted/Unweighted, Cyclic/Acyclic.

Data Structures for Graph Representation

- **Adjacency Matrix:** A 2D array $Adj[V][V]$. $Adj[i][j] = 1$ (or weight w) if an edge exists from i to j , else 0 . Space Complexity is $O(V^2)$. Best for dense graphs. Finding an edge is $O(1)$.
- **Adjacency List:** An array of Linked Lists of size V . Each vertex has a linked list containing all its adjacent vertices. Space Complexity is $O(V + E)$. Best for sparse graphs. Finding an edge takes $O(V)$ in worst case.

2. Graph Traversal Algorithms

Breadth First Search (BFS)	Depth First Search (DFS)
Explores level by level. Visits all immediate neighbors first.	Explores branch by branch. Goes as deep as possible before backtracking.
Uses a Queue data structure.	Uses a Stack (or System call stack via recursion).
Finds the shortest path on unweighted graphs. Time Complexity: $O(V+E)$.	Used for Topological Sorting, finding connected components, cycle detection. Time Complexity: $O(V+E)$.

3. Minimum Cost Spanning Trees (MST)

A **Spanning Tree** of a connected, undirected graph is a subgraph that is a tree and includes all the vertices. A Minimum Spanning Tree has the minimum possible total edge weight. For V vertices, an MST will have exactly $V-1$ edges.

- **Kruskal's Algorithm:** A Greedy algorithm.

1. Sort all edges in non-decreasing order of weight.
2. Pick the smallest edge. If adding it to the MST does not form a cycle (checked using Disjoint Set Union-Find), include it.
3. Repeat until $V-1$ edges are added.

Time Complexity: $O(E \log E)$ or $O(E \log V)$.

- **Prim's Algorithm:** A Greedy algorithm.

1. Start with an arbitrary vertex, add it to the MST set.
2. Find the minimum weight edge connecting a vertex in the MST set to a vertex outside the MST set.
3. Add the edge and the new vertex to the MST set.
4. Repeat until all vertices are included.

Time Complexity: $O(E \log V)$ using an Adjacency List and Min-Heap.

4. Shortest Path Algorithms

Dijkstra's Algorithm (Single Source Shortest Path)

Finds the shortest paths from a single source vertex to all other vertices in a weighted graph with **non-negative** weights.

Algorithm: Maintains a set of visited vertices and an array of distances from the source (initialized to ∞). Iteratively picks the unvisited vertex with the minimum distance, marks it visited, and *relaxes* all its adjacent unvisited edges.

Time Complexity: $O((V+E) \log V)$ with a Min-Priority Queue.

Floyd-Warshall Algorithm (All Pairs Shortest Path)

Finds the shortest paths between all pairs of vertices in a weighted graph. Uses Dynamic Programming.

Algorithm: It initializes a distance matrix with edge weights. It iteratively checks if a path passing through an intermediate vertex k is shorter than the current known path.

Core Relation:

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$$

Time Complexity: $O(V^3)$.

Transitive Closure and Warshall's Algorithm

Transitive Closure determines if there is a valid path from vertex i to vertex j for all possible pairs (i, j) in a directed graph. Warshall's algorithm is identical in structure to Floyd-Warshall but uses boolean logic (AND/OR) instead of arithmetic addition/minimum to build a reachability matrix.